

Project-3 Report

Name – Surname: Barış Kaplan

Matriculation Number: 669584

In this project, the following class mappings are used:

Class 0 => "World"

Class 1 => "Sports"

Class 2 => "Business"

Class 3 => "Sci/Tech"

Task 1:

Used Classification Model: Logistic Regression

Classifier Baseline Test Accuracy Achieved: 90.55%

```
# Split the imported data into train and test parts
self.X_train_raw = dataset['train']['text']
self.y_train = np.array(dataset['train']['label'])
self.X_test_raw = dataset['test']['text']
self.y_test = np.array(dataset['test']['label'])

# Text Feature Extraction
self.vectorizer = TfidfVectorizer(max_features=5000, stop_words='english')
self.X_train = self.vectorizer.fit_transform(self.X_train_raw).toarray()
self.X_test = self.vectorizer.transform(self.X_test_raw).toarray()

# Baseline Classifier
self.baseline_model = LogisticRegression(C=0.4, max_iter=200)
self.baseline_model.fit(self.X_train, self.y_train)
self.baseline_test_preds = self.baseline_model.predict(self.X_test)
self.baseline_acc = float(accuracy_score(self.y_test, self.baseline_test_preds))
```

Task 2:

Simulated Expert's Test Dataset Accuracy Achieved: 92.70% (Specialized expertise in Class 1 (Sports))

```
# Simulated Expert
self.expert_test_preds = self.simulate_expert_predict(self.y_test)
self.expert_acc = float(accuracy_score(self.y_test, self.expert_test_preds))

def simulate_expert_predict(self, y_true):
    """ Simulates bounded human expert decisions. """
    expert_preds = []
    for label in y_true:
        # Specific domain specialization
```

```

if label == 1:
    expert_preds.append(label)
else:
    if np.random.rand() < 0.90:
        expert_preds.append(label)
    else:
        remaining_classes = [c for c in range(4) if c != label]
        expert_preds.append(np.random.choice(remaining_classes))
return np.array(expert_preds)

```

Strengths of the Simulated Expert on the Dataset

1. Domain Expertise in Specific Regions of the Input Space: This simulated expert exhibits a deterministic behavior for Class 1, which is the “Sports” field. It suitably mimics the knowledge of real-world human experts who possess deep knowledge in a specific field (Class 1: Sports), instead of holding a generalized and/or broad expertise. Therefore, it will serve as a realistic and resource-efficient proxy substitute to evaluate cost-efficient (efficient in terms of time, computational complexity, and financial budget) routing and decision delegation mechanisms within the learning-to-defer framework.

2. High Accuracy Achieved on the Test Dataset: The simulated expert has achieved a high test dataset accuracy of “92.70%”, which means it is a highly reliable ground-truth substitute.

3. Significant Reduction of Time, Effort, and Cost Needed for Manual Data Labeling: This simulated expert can serve as a highly reliable proxy for a manual domain expert workforce on the ground-truth dataset; as a result, this approach will significantly reduce the time, effort, and cost required for manual data labeling.

Weaknesses of the Simulated Expert on the Dataset

1. Uniform Error Distribution: The simulated expert makes mistakes %10 of his/her choices. A class for the given text is randomly and uniformly chosen from the remaining options, with equal probability for each class option. However, this is not the case for the decisions of real-life human experts. Human experts rarely guess uniformly and randomly. They are typically influenced by cognitive biases, systematic biases, patterned behavior, and social context, which makes them usually confuse specific classes sharing similar features.

```

else:
    if np.random.rand() < 0.90:
        expert_preds.append(label)
    else:
        remaining_classes = [c for c in range(4) if c != label]
        expert_preds.append(np.random.choice(remaining_classes))

```

2. Class Imbalance Problem: When human experts encounter an unclear text, they will inherently develop a high level of prior bias towards the classes with more dominance than the others. However, my simulated expert cannot observe the frequencies of the classes (labels). Therefore, its guessing behavior and following prediction errors do not change based on which classes are seen more often, which means the simulated expert is not affected by the class distribution.

3. Static Error Ratio: The error ratio is static and 10% for all remaining classes except Class 1, which are Class 0, 2, and 3. However, the human experts' data labeling performance and errors made in this process change a lot based on several factors, such as tiredness level, daytime, and changing expertise levels for the remaining classes.

4. No Contextual Knowledge of the Input X Values: Even though the simulated expert knows the ground-truth class labels (Class 0 => "World", Class 1 => "Sports", Class 2 => "Business", Class 3 => "Sci/Tech"), it does not know how the features of the input values differ from each other in terms of the length, number, meaning, complexity, and structure of the words in each input text.

5. No “I am completely unsure” Option: This expert is forced to select from the available labels in the error case. However, our expert in real-life might not be able to classify the input into the provided categories and put it into a category like "Unknown".

Task 3:

Task 4:

Task 5: